

Phase 3: Finalization Abstract

Project: Shopify-based jewelry inventory identification and tagging automation

Alp Ege Yalcin | Matriculation no. 42308653 | DLBCCOEDAH01 Project: Digitalization and Automation Hackathon

Source code and implementation evidence:

github.com/ApteryxDev/Project-Digitalization-and-Automation-Hackathon-DLBCCOEDAH01- (see Appendix C)

1. Objective and final solution

The project started from a process I had observed while working in a family-owned jewelry shop: identifying individual pieces, finding the matching product in Shopify, checking stock and preparing a small jewelry tag or a box label. None of these steps is complex on its own, but repeating them for many products made the workflow slow and prone to avoidable mistakes. The final prototype is a Python/Flask web application that links the physical inventory to Shopify through the eight-digit barcodes already used in the shop. After a barcode is scanned or typed, the application retrieves the product, image, price and inventory, and from the same screen the user can prepare a stock adjustment, generate PDF labels and view a restocking suggestion. A Demo Mode with fictional products allows the workflow to be shown without access to the shop's account. One rule was kept throughout: an inventory change is never published automatically; the user has to confirm it first.

2. Evaluation against the KPIs (As-Is versus To-Be)

Appendix A compares the manual and prototype-assisted workflow for the five KPIs defined in Phase 1. For item identification, an illustrative estimate based on the implemented workflow suggests roughly 25 to 45 seconds per item when searching by name in Shopify admin, against about 4 to 8 seconds with a barcode scan. For a batch of 20 items, this corresponds to roughly 10 to 14 minutes compared with about 2 minutes. Preparing ten tags by hand was estimated at 8 to 12 minutes, against 2 to 3 minutes for selecting the items and downloading one PDF. Manual lookups after scanning were close to zero when a barcode exists. These figures are estimates derived from the workflow, not results of a controlled time study, and I did not estimate the inventory-record error rate. Low-stock detection changes qualitatively from periodic manual review to a value visible on demand. A timed comparison on the same set of items is therefore the first activity of the pilot described below.

3. Implementation and feasibility

I tested the application first with fictional products and then with real Shopify records and physical items. The second stage exposed problems the concept had not anticipated: SKU fields were often empty while barcodes were consistently available, so the barcode became the primary key; inventory locations and API scopes had to be handled explicitly rather than guessed; and the printer-specific label output was replaced by PDFs that can be previewed before printing. The current-stock value in the restocking view, reported as unreliable in API mode in Phase 2, remains a known limitation that must be resolved before the restocking rule is used operationally. Technically, the solution is feasible for the shop: it runs on an existing computer and browser, needs only barcode input and the shop's printers, and keeps Shopify as the single system of record.

4. Resources, cost and business impact

Appendix B summarizes the required resources. The software stack (Python, Flask and ReportLab) is open source and introduces no licence cost, and no second inventory database is needed. The

main costs are time: deployment, printer calibration, permission setup, maintenance and staff training. I cannot give a defensible return on investment without a measured baseline, but the estimates above show where the potential value lies: fewer repeated searches, faster and more consistent label preparation, clearer stock updates and earlier visibility of items that may need reordering. Schlegel et al. (2024) note that automation projects benefit from stable and rule-based processes, which is consistent with the functions implemented here.

5. Scalability and continuous improvement

The code is split into services for Shopify access, demo data, inventory logic and label generation, so individual parts can be replaced without rewriting the prototype. The next improvements, in order of practical value, are user authentication, stronger multi-location handling, automatic low-stock notifications, scan-history analytics and supplier or purchase-order integration. For restocking I deliberately used a transparent rule (average monthly sales over six months, three months of coverage, a 1.20 safety factor, minus current stock) instead of presenting it as an AI prediction, because the user can understand and check the calculation. Once enough sales history exists, this rule could be compared with a seasonal or time-series forecast to determine whether a more advanced method is actually justified (Jasińska et al., 2023). Success would then be evaluated by repeating the KPI measurements at fixed intervals during the pilot.

6. Critical reflection and recommendation

One of the main lessons from the project was that the most useful solution was not necessarily the most technically complex one. Although this project sits within my Applied Artificial Intelligence studies, a machine-learning component would not have solved the actual problem more effectively. The practical challenge was connecting physical items reliably to data that already existed in Shopify and producing outputs that could actually be used in the shop. I therefore recommend a limited pilot alongside the existing workflow rather than an immediate full rollout. The pilot should calibrate the labels on the final printers, verify permissions and locations, fix the current-stock retrieval issue and record the KPI measurements for a defined set of items. The decision on wider deployment should then be based on those measurements rather than on the estimates in this abstract.

References

- Jasińska, K., Lewicz, M., & Rostalski, M. (2023). Digitization of the enterprise: Prospects for process automation with using RPA and GPT integration. *Procedia Computer Science*, 225, 3243–3254.
- Schlegel, D., Rosenberg, B., Fundanovic, O., & Kraus, P. (2024). How to conduct successful business process automation projects? An analysis of key factors in the context of robotic process automation. *Business Process Management Journal*, 30(8), 99–119.

Supplementary Appendix

Appendices A to C are supplementary material and are not part of the two-page Finalization Abstract. They document the KPI assumptions, pilot resources and project repository referred to in the abstract.

Appendix A: KPI evaluation, As-Is versus To-Be

KPI (Phase 1)	As-Is (manual workflow)	To-Be (prototype)	Illustrative estimate	Status
Item identification time	Search by name in Shopify admin: approx. 25–45 s per item (10–14 min for 20 items)	Scan or type the 8-digit barcode: approx. 4–8 s per item (about 2 min for 20 items)	Approx. 75–85% faster; 8–12 min saved per 20 items	Estimated from workflow; to be timed in pilot
Tag generation time	Tags prepared or matched by hand: approx. 4–6 min for 5 tags, 8–12 min for 10	Select items, download one PDF: approx. 1–2 min for 5 tags, 2–3 min for 10	Approx. 60–80% faster	Estimated from workflow; to be timed in pilot
Manual lookups per item	Every item requires a manual search	Approx. 0–1 manual lookups per 20 items (only when a barcode is missing)	Largely eliminated where a barcode exists	Observed during testing
Inventory-record error rate	Manual matching and typing can select the wrong item or quantity	Barcode validation plus a separate Publish step	No estimate made	Not measured; requires a physical count
Low-stock detection lag	Noticed only at periodic manual review	Target stock and order suggestion visible on demand	Qualitative improvement	Restocking view limited by current-stock issue (Phase 2)

Table A1. Comparison of the manual and prototype-assisted workflow for the five Phase 1 KPIs. Time values are illustrative estimates derived from the implemented workflow and are to be validated by timed testing in the shop.

Appendix B: Resources and cost

Resource	Assessment
Software	Python, Flask and ReportLab; open-source components, no new proprietary licence.
Existing system	Shopify remains the system of record; no second production inventory database.
Hardware	Existing computer and browser, barcode scanner or keyboard input, and the shop's label and A4 printers.
Implementation effort	Deployment, testing, printer calibration, permissions and locations, maintenance, staff and developer time.
Business impact	Operational: fewer repeated searches, consistent labels, clearer stock updates and earlier visibility of restocking needs.

Table B1. Resources required for a pilot deployment.

Appendix C: Project repository and implementation evidence

The complete source code of the prototype is available in the public project repository:

<https://github.com/ApteryxDev/Project-Digitalization-and-Automation-Hackathon-DLBCCOEDAH01->. See the folder "Pictures of implementation" in the repository for screenshots, generated labels and physical implementation evidence.